UTN
Facultad Regional Córdoba

TUP

Tecnicatura Universitaria en Programación

LABORATORIO DE COMPUTACIÓN IV

Unidad Temática N°2:

Docker

Guía Práctica

2° Año – 4° Cuatrimestre

CC BY ND NC

Índice

Docker CLI 2

Comandos de Docker CLI 2

Verificación de Instalación 2
Descargar Imágenes de Docker - Docker pull 2
Correr un contenedor - Docker run 5
Variables de entorno 6
Mapear puertos 6
Asignar un volumen 7
Correr en modo detach 7
Manejar Contenedores - Docker container 8
Ciclo de vida de los contenedores 9
Logs de los contenedores 10

Crear una Imagen de Docker 11

Misceláneo 11

Ejercitación 13

Bibliografía 14

Docker CLI

Docker Command-Line Interface (CLI), también conocida como **Docker CLI** o Docker Engine CLI, *es la herramienta que nos permite interactuar con Docker mediante comandos en la terminal*. Proporciona una serie de instrucciones fáciles de usar para gestionar contenedores, imágenes, redes y volúmenes, lo que permite a los desarrolladores y administradores trabajar con Docker de manera eficiente y efectiva.

Comandos de Docker CLI

Verificación de Instalación

Una vez que hayas instalado **Docker** correctamente en tu sistema operativo, deberías poder ejecutar los comandos de **Docker CLI** para gestionar sus diversos componentes. En primer lugar, es recomendable asegurarse de que esté configurado correctamente utilizando el siguiente comando:

```
docker --version
```

Descargar Imágenes de Docker - Docker pull

Si necesitamos descargar una imagen de docker podemos hacerla desde el **Docker Registry**, que es un repositorio público de imágenes de todo tipo. Para acceder al mismo entramos a https://hub.docker.com/ . Allí puedes encontrar imágenes de todos tipos que resuelven diversos problemas.

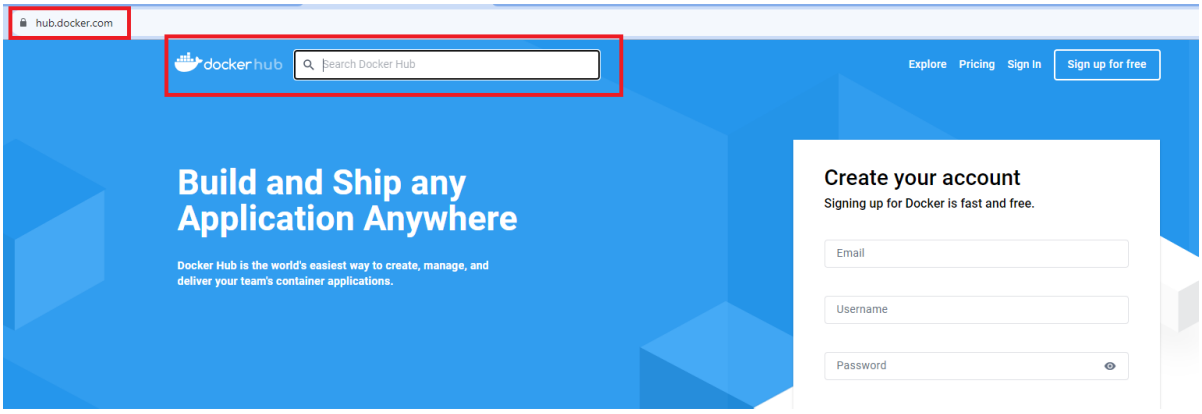

Imagen 1: Elaboración propia.

Supongamos que queremos correr una base de datos en nuestra máquina, para esto podríamos instalar todo un motor de base de datos pero sería un proceso engorroso y además si quisiésemos tener varias instancias en nuestra maquina no podríamos hacerlo.

Para eso Docker nos va a ayudar, podemos correr en un contenedor una instancia de una BD y la podemos frenar, reiniciar a nuestro antojo. Es más, podríamos tener muchas instancias corriendo al mismo tiempo sin ningún problema. Para ello, podemos buscar en Docker Hub y encontrar la página oficial de la imagen de PostgreSQL. Se visualizará como en la siguiente imagen:

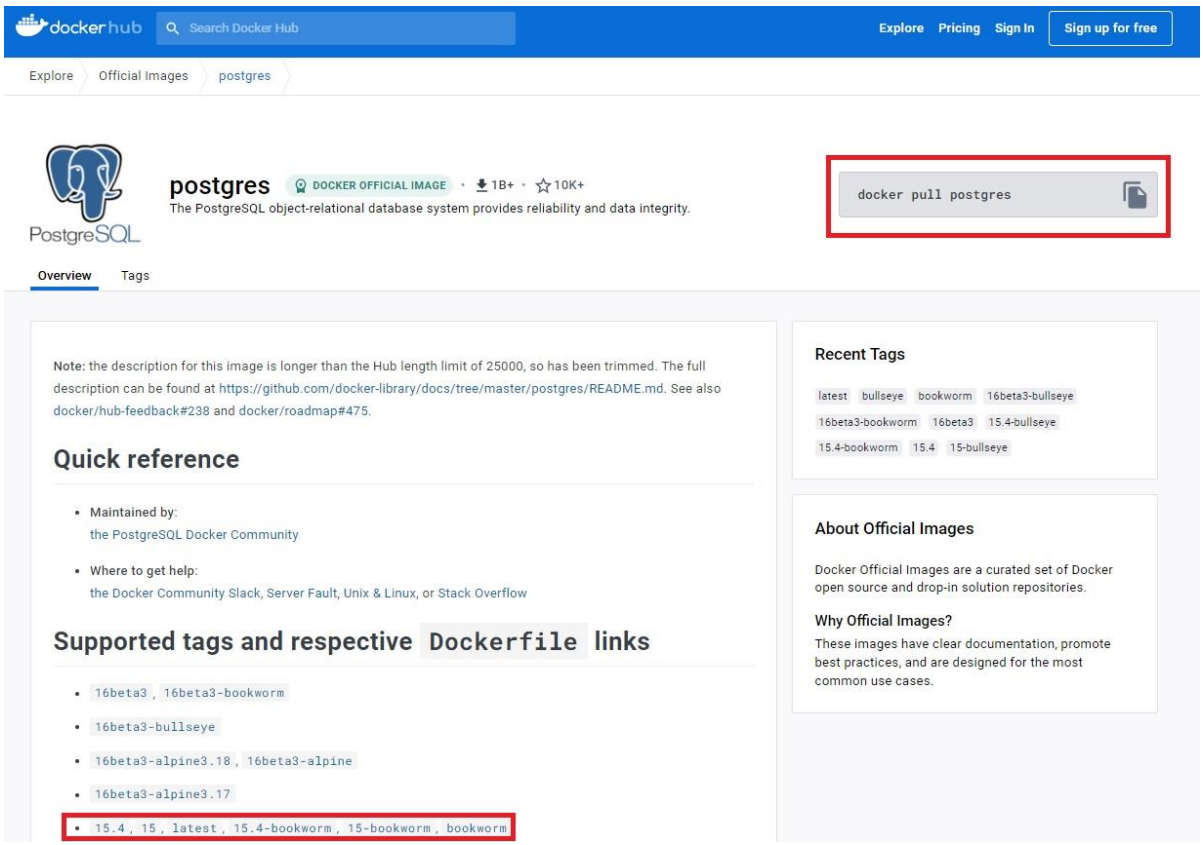

Imagen 2: Elaboración propia.

Podemos ver que en la parte superior nos indica el comando para descargar la imagen y en la parte superior los distintos tags que hacen referencia a las versiones disponibles de esta imagen, los tags son etiquetas que identifican a las versiones de las imágenes.

Además, más abajo muestra configuraciones extra que podemos definir al correr esta imagen en un contenedor.

Por lo tanto, si queremos descargar la imagen, se utiliza el siguiente comando:

```
docker pull NOMBRE_DE_LA_IMAGEN
```

En este ejemplo utilizamos PostgreSQL:

```
docker pull postgres
```

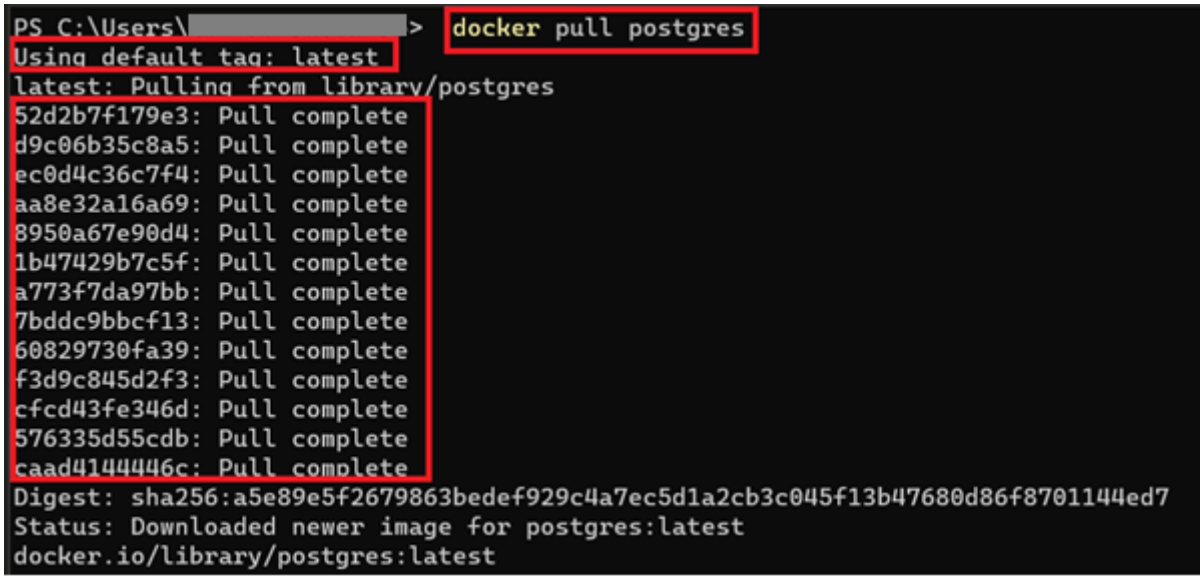

Imagen 3: Elaboración propia.

Podemos ver que este comando descarga la última versión disponible de esa imagen que se encuentra en el registro (Docker Hub). Si observamos en la salida del comando, se utiliza por defecto el tag "latest" si no le especificamos la versión que deseamos usar.

Además, vemos como descarga las distintas capas que componen esta imagen.

Si quisiéramos especificar una versión que nos interesa, debemos indicarla después del nombre, separada por dos puntos:

```
docker pull NOMBRE_DE_LA_IMAGEN:NUMERO_DE_VERSION
```

Ejemplo:

```
docker pull postgres:15
```

Es importante destacar que una imagen puede estar asociada con más de un tag. Por lo tanto, en el ejemplo anterior, la versión 15 tiene el tag *latest*. Por lo tanto, ambos tags descargaron la misma imagen.

Correr un contenedor - Docker run

Para iniciar un contenedor, simplemente utiliza el siguiente comando:

```
docker run NOMBRE_DE_LA_IMAGEN
```

Esto crea un contenedor a partir de la imagen seleccionada con su configuración predeterminada.

NOTA: Al igual que con el comando para descargar imágenes, Docker nos permite **especificar la versión** que queremos utilizar de la imagen seleccionada.

```
docker run NOMBRE_DE_LA_IMAGEN:NUMERO_DE_VERSION
```

Ejemplo:

```
docker run postgres
```

Cuando ejecutamos este comando vemos que nos devuelve un mensaje de error indicando de que debemos especificar la contraseña del usuario root via variable de entorno.

```
PS C:\Users\[redacted]> docker run postgres
Error: Database is uninitialized and superuser password is not specified.
       You must specify POSTGRES_PASSWORD to a non-empty value for the
       superuser. For example, "-e POSTGRES_PASSWORD=password" on "docker run".

       You may also use "POSTGRES_HOST_AUTH_METHOD=trust" to allow all
       connections without a password. This is *not* recommended.

       See PostgreSQL documentation about "trust":
       https://www.postgresql.org/docs/current/auth-trust.html
```

Imagen 4: Elaboración propia.

Pero ¿qué son las variables de entorno?

Variables de entorno

En algunos casos, es necesario asignar configuraciones adicionales a un contenedor, para esto utilizamos estas variables de entorno. Estas variables son cargadas al ejecutar el contenedor y modifican su comportamiento y permitiéndonos personalizar el funcionamiento de este.

Muchas imágenes requieren ciertas configuraciones para su funcionamiento, las cuales deben ser transmitidas a través de variables de entorno, accesibles desde el sistema operativo.

```
docker run -e VAR_1=ejemplo NOMBRE_DE_LA_IMAGEN
```

Entonces para nuestro ejemplo de postgres deberíamos ejecutar el comando de esta manera:

```
docker run -e POSTGRES_PASSWORD=secret -e POSTGRES_USER=postgres
postgres
```

Ya tenemos corriendo nuestra base de datos en nuestra computadora, las variables de entorno que le configuramos definen el usuario y contraseña del usuario root de la BD.

Mapear puertos

Hasta ahora, nuestros contenedores funcionan sin problemas, pero no podemos interactuar con ellos porque están aislados en su propia red. Para permitir la interacción vía red con un contenedor, es necesario mapear un puerto del contenedor al puerto de la máquina anfitriona que está ejecutando el contenedor. De esta manera, al conectarnos a ese puerto en el anfitrión, nos conectamos al contenedor.

```
docker run -p PUERTO_EXTERNO:PUERTO_INTERNO NOMBRE_DE_LA_IMAGEN
```

Ejemplo:

```
docker run -p 5432:5432 postgres
```

En caso de que el puerto externo ya esté en uso, podemos mapearlo a otro puerto en nuestra máquina:

```
docker run -p 55432:5432 postgres
```

Asignar un volumen

Hasta este punto, tenemos contenedores ejecutándose con las imágenes deseadas, pero cualquier dato guardado en un contenedor se perderá al reiniciar. Esto se debe a que los contenedores están diseñados para contener solo la información incluida en el momento de creación de la imagen y mantener un comportamiento constante en cada ejecución.

Para permitir que los contenedores conserven datos, es necesario adjuntar un volumen al contenedor. Un volumen es una referencia a un espacio en disco en el host que puede ser utilizado por uno o varios contenedores.

Si deseamos que el volumen enlazado al contenedor esté relacionado con una ruta específica en nuestra máquina, se logra con el comando:

```
docker run -v RUTA_ORIGEN_HOST:RUTA_DESTINO_CONTENEDOR
NOMBRE_DE_LA_IMAGEN
```

Ejemplo:

```
docker run -e POSTGRES_PASSWORD=secret -e POSTGRES_USER=postgres
           -v ./postgres-data:/var/lib/postgresql/data  postgres
```

Este comando crea un volumen en la ruta que definimos y se lo asigna al contenedor, permitiendo que los datos que se persistan queden guardados en ese volumen y no se pierda la información si reiniciamos el contenedor.

Correr en modo detach

En ocasiones, es necesario que la ejecución del contenedor no esté vinculada a la sesión de la terminal, lo que significa que el contenedor continuará ejecutándose incluso después de cerrar la terminal. Para lograr esto, simplemente agrega la variable "detach" al ejecutar el contenedor:

```
docker run -d -e POSTGRES_PASSWORD=secret
              -e POSTGRES_USER=postgres postgres
```

```
PS C:\Users\[redacted]> docker run -d -e POSTGRES_PASSWORD=secret -e POSTGRES_USER=postgres postgres
aafbf9f18674cd3288604ce164a374f30a11f84b5cf47e9de8c4e019a3040bbb
PS C:\Users\Daniel Cattaneo> docker container ls
CONTAINER ID   IMAGE      COMMAND                  CREATED         STATUS         PORTS      NAMES
aafbf9f18674   postgres   "docker-entrypoint.s…"   4 seconds ago   Up 3 seconds   5432/tcp   sweet_wilson
PS C:\Users\[redacted]>
```

Imagen 5: Elaboración propia.

Es importante ver que cuando ejecutamos en modo detach el comando nos devuelve simplemente el id completo del contenedor y si luego listamos los contenedores corriendo podemos acceder a el y ver la información del mismo.

Manejar Contenedores - Docker container

Una vez que tengamos nuestros contenedores en funcionamiento, es posible que necesitemos supervisar su estado y realizar modificaciones. Para listar los contenedores en ejecución, simplemente utiliza:

```
docker container ls
```

```
PS C:\Users\[redacted]> docker container ls
CONTAINER ID   IMAGE      COMMAND                  CREATED         STATUS         PORTS      NAMES
31c949d492d1   postgres   "docker-entrypoint.s…"   7 seconds ago   Up 6 seconds   5432/tcp   pedantic_merkle
```

Imagen 6: Elaboración propia.

Este comando nos muestra el identificador de nuestro contenedor, así como la imagen que está corriendo, hace cuanto fue creado, su estado y nos indica si tiene puertos mapeados.

Se puede observar que muestra únicamente los contenedores en funcionamiento. Si deseamos ver todos los contenedores, incluidos los que ya no están en ejecución, podemos agregar la flag -a:

```
docker container ls -a
```

```
PS C:\Users\[redacted]> docker container ls -a
CONTAINER ID   IMAGE      COMMAND                  CREATED          STATUS                      PORTS      NAMES
31c949d492d1   postgres   "docker-entrypoint.s…"   2 minutes ago    Up 2 minutes                5432/tcp   pedantic_merkle
3f1b6d9023b8   postgres   "docker-entrypoint.s…"   4 minutes ago    Exited (0) 2 minutes ago               objective_ritchie
6593752f8c39   postgres   "docker-entrypoint.s…"   7 minutes ago    Exited (1) 7 minutes ago               pedantic_bohr
6d4e539c713c   busybox    "sh"                     19 minutes ago   Exited (0) 19 minutes ago              flamboyant_moser
36c14cf17183   ubuntu     "bash"                   20 minutes ago   Exited (0) 20 minutes ago              lucid_turing
b739cee246af   ubuntu     "/bin/bash"              20 minutes ago   Exited (0) 20 minutes ago              great_germain
PS C:\Users\[redacted]>
```

Imagen 7: Elaboración propia.

De este modo podemos observar los otros contenedores que hemos corrido y ***ya no están ejecutándose***.

Ciclo de vida de los contenedores

Por ejemplo, para pausar la ejecución de un contenedor, utiliza:

```
docker container pause CONTAINER_ID
```

```
PS C:\Users\[redacted]> docker container ls -a
CONTAINER ID   IMAGE      COMMAND                  CREATED          STATUS                      PORTS      NAMES
31c949d492d1   postgres   "docker-entrypoint.s…"   2 minutes ago    Up 2 minutes                5432/tcp   pedantic_merkle
3f1b6d9023b8   postgres   "docker-entrypoint.s…"   4 minutes ago    Exited (0) 2 minutes ago               objective_ritchie
6593752f8c39   postgres   "docker-entrypoint.s…"   7 minutes ago    Exited (1) 7 minutes ago               pedantic_bohr
6d4e539c713c   busybox    "sh"                     19 minutes ago   Exited (0) 19 minutes ago              flamboyant_moser
36c14cf17183   ubuntu     "bash"                   20 minutes ago   Exited (0) 20 minutes ago              lucid_turing
b739cee246af   ubuntu     "/bin/bash"              20 minutes ago   Exited (0) 20 minutes ago              great_germain
PS C:\Users\[redacted]> docker container pause 31c949d492d1
31c949d492d1
PS C:\Users\[redacted]> docker container ls -a
CONTAINER ID   IMAGE      COMMAND                  CREATED          STATUS                      PORTS      NAMES
31c949d492d1   postgres   "docker-entrypoint.s…"   3 minutes ago    Up 3 minutes (Paused)       5432/tcp   pedantic_merkle
3f1b6d9023b8   postgres   "docker-entrypoint.s…"   5 minutes ago    Exited (0) 3 minutes ago               objective_ritchie
6593752f8c39   postgres   "docker-entrypoint.s…"   8 minutes ago    Exited (1) 8 minutes ago               pedantic_bohr
6d4e539c713c   busybox    "sh"                     20 minutes ago   Exited (0) 20 minutes ago              flamboyant_moser
36c14cf17183   ubuntu     "bash"                   21 minutes ago   Exited (0) 21 minutes ago              lucid_turing
b739cee246af   ubuntu     "/bin/bash"              22 minutes ago   Exited (0) 21 minutes ago              great_germain
PS C:\Users\[redacted]>
```

Imagen 8: Elaboración propia.

Podemos ver que al pausar el contenedor su estado cambia y se ve reflejado cuando volvemos a listar los contenedores corriendo.

Si luego queremos reanudar su ejecución podemos ejecutar:

```
docker container unpause CONTAINER_ID
```

Si queremos detener un contenedor, utilizamos:

```
docker container stop CONTAINER_ID
```

En casos donde necesitemos forzar el cierre de un contenedor, utilizamos:

```
docker container kill CONTAINER_ID
```

También podemos reiniciar la ejecución de un contenedor mediante:

```
docker container restart CONTAINER_ID
```

Logs de los contenedores

Si necesitamos acceder a los logs de un contenedor podemos hacerlo con:

```
docker logs CONTAINER_ID
```

Este comando muestra todos los logs desde la creación del contenedor. Sin embargo, esto puede ser problemático cuando solo necesitamos ver una parte más reciente de estos. Para esto, podemos utilizar la flag -n seguida del número de líneas que queremos visualizar:

```
docker logs -n CANT_LINEAS CONTAINER_ID
```

Ejemplo:

```
docker logs -n 5 82fa5d18bdd8
```

```
PS C:\Users\[redacted]> docker container logs -n 5 aafbf9f18674
2023-08-28 18:30:57.658 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
2023-08-28 18:30:57.659 UTC [1] LOG:  listening on IPv6 address "::", port 5432
2023-08-28 18:30:57.667 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2023-08-28 18:30:57.678 UTC [62] LOG:  database system was shut down at 2023-08-28 18:30:57 UTC
2023-08-28 18:30:57.687 UTC [1] LOG:  database system is ready to accept connections
PS C:\Users\[redacted]>
```

Imagen 8: Elaboración propia.

Es importante señalar que este número se cuenta desde las líneas de los logs más recientes hacia los más antiguos.

También puede ser necesario "seguir" los logs mientras se generan, lo que significa que los nuevos logs se irán mostrando a medida que se generen. Para lograr esto, utilizamos el comando:

```
docker container -f CONTAINER_ID
```

```
PS C:\Users\[redacted]> docker container logs -n 5 -f aafbf9f18674
2023-08-28 18:30:57.658 UTC [1] LOG:  listening on IPv4 address "0.0.0.0", port 5432
2023-08-28 18:30:57.659 UTC [1] LOG:  listening on IPv6 address "::", port 5432
2023-08-28 18:30:57.667 UTC [1] LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2023-08-28 18:30:57.678 UTC [62] LOG:  database system was shut down at 2023-08-28 18:30:57 UTC
2023-08-28 18:30:57.687 UTC [1] LOG:  database system is ready to accept connections
```

Imagen 9: Elaboración propia.

Podemos ver que no podemos escribir un nuevo comando porque el modo follow va a seguir mostrando los logs a medida que se van generando. Si queremos salir de este comando usamos **Control+C**.

Crear una Imagen de Docker

Misceláneo

Existen algunos comandos adicionales que pueden ayudarnos a diagnosticar posibles problemas.

Por ejemplo, para conocer el consumo de recursos de nuestros contenedores, podemos utilizar:

```
docker stats
```

```
CONTAINER ID   NAME           CPU %     MEM USAGE / LIMIT     MEM %     NET I/O       BLOCK I/O   PIDS
aafbf9f18674   sweet_wilson   0.00%     31.12MiB / 7.613GiB   0.40%     1.09kB / 0B   0B / 0B     6
```

Imagen 10: Elaboración propia.

Esta información nos puede servir para monitorear nuestros contenedores, ver si realmente están ejecutando algo, ver si los recursos asignados son suficientes, ver si están recibiendo tráfico de red, etc.

También puede surgir la necesidad de ejecutar comandos dentro de un contenedor a través de la terminal. Para esto, utilizamos:

```
docker exec -it CONTAINER_ID bash
```

Finalmente, es importante destacar que después de usar Docker en repetidas ocasiones, nuestro disco podría llenarse con imágenes que ya no utilizamos o versiones antiguas de las imágenes. Para abordar este problema, Docker proporciona varios comandos para la limpieza de sus diferentes componentes:

```
docker image prune
docker container prune
```

Estos comandos eliminan imágenes no utilizadas y contenedores finalizados.

Lo mismo se aplica a los volúmenes que creamos. Si necesitas eliminar un volumen, puedes hacerlo con el comando:

```
docker volume rm VOLUME_ID
```

Finalmente, puede surgir la necesidad de acceder a un contenedor a través de la línea de comandos y ejecutar comandos dentro de él. Para lograr esto, puedes utilizar:

```
docker exec -it CONTAINER_ID bash
```

Ejercitación

A continuación, se presenta una lista de ejercicios con el propósito de poner en práctica todos los comandos vistos en este documento:

1. Investigar para que se usa la imagen de **nginx** y levantar un contenedor de esa imagen.
2. Mapear los puertos de ese contenedor, de manera tal de que puedas acceder desde el navegador.
3. Levantar dos instancias distintas de **nginx** en puertos distintos.
4. Revisar los logs del contenedor.
5. Pausar uno de los dos contenedores.
6. Reiniciar la ejecución del contenedor pausado.
7. Reiniciar ambos contenedores.
8. Frenar la ejecución de ambos contenedores y eliminar los contenedores.
9. Levantar un contenedor de **MySQL** con la versión 8.0.34.
10. Correr el contenedor con el puerto 3306 mapeado al puerto 3366 de tu máquina.
11. Investigar cómo configurar la contraseña del usuario root al correr el contenedor.
12. Correr el contenedor asociando un volumen.
13. Reiniciar el contenedor y asegurarse que los datos guardados sigan presentes.

Bibliografía

- Use the Docker Command Line:

  https://docs.docker.com/engine/reference/commandline/cli/

Atribución-No Comercial-Sin Derivadas

Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.